LAPORAN TUGAS KELOMPOK 3

Analisis dan Perancangan Arsitektur Layanan Tugas pada SPADA UMI-Kalam
(Subsistem Assignment & Assessment)

NAMA ANGGOTA KELOMPOK :

NUR FADHILAH FAUZIAH – 13020240251

NUR AFIFAH – 13020240030

FERA JULIA HALIM – 13020240329

ST. NABILA AZIRA ZAMZANI A – 13020240324

Mata Kuliah	Arsitektur Perangkat Lunak
Dosen Pengampu	Mardiyyah Hasnawi, S.Kom., M.T.
Program Studi	Teknik Informatika
Kelompok	Bagian I - Kelompok 3
Tahun Akademik	2026

DAFTAR ISI

LAPORAN TUGAS KELOMPOK I 1
DAFTAR ISI 1
BAB I 4
PENDAHULUAN 4
1.1 Ringkasan Layanan dan Pembaharuan 4
1.2 Masalah Utama Layanan dan Bukti Nyata 4
1.3 Stakeholder Utama 5
1.4 Kebutuhan Signifikan Arsitektur (ASR) 6
1.5 Keterkaitan Quality Drivers dengan ASR 6
BAB II 7
SKENARIO 7
2.1 Skenario 1: Performance Efficiency (Menjawab Masalah Utama) 7
2.2 Skenario 2: Security & Integrity (Keamanan & Integritas Data) 7
2.3 Skenario 3: Maintainability (Kemudahan Pemeliharaan) 8
BAB III 9
TAKTIK DAN POLA ARSITEKTUR 9
3.2 Taktik keamanan: Authentication dan Authorization 9
3.3 Taktik ketersediaan: Redundancy dan Retry Mechanism 10
4.1 Pola Layered Architecture 11
4.2 Pola Event-Driven Architecture 12
4.3 Perbedaan kedua pola 13
Aspek 13
Layered Architecture 13
Event-Driven Architecture 13
Fokus 13
Pembagian tanggung jawab sistem 13
Komunikasi berdasarkan peristiwa 13
Tujuan 13
Membuat struktur sistem terorganisasi 13
Memisahkan proses dan mengurangi ketergantungan langsung 13
Penerapan 13
Antarmuka, logika bisnis, akses data, database 13
Pengumpulan tugas, antrean, pemeriksaan kemiripan, notifikasi 13
Masalah yang dibantu 13
Kompleksitas struktur aplikasi 13
Lonjakan proses dan pekerjaan yang membutuhkan waktu lama 13

BAB IV 14
DESAIN 14
5.1 Diagram konteks 14
Diagram konteks menunjukkan hubungan antara sistem dengan pihak-pihak yang berinteraksi dengannya. 14
5.2 Diagram arsitektur yang menerapkan pola 15
Diagram berikut menggabungkan Layered Architecture dan Event-Driven Architecture. Layered Architecture digunakan untuk mengatur tanggung jawab aplikasi, sedangkan Event-Driven Architecture digunakan untuk menjalankan pemeriksaan kemiripan secara asynchronous. 15
5.3 Diagram UML (Sequence Diagram) 16
Diagram UML berikut menggunakan Sequence Diagram untuk menunjukkan urutan interaksi ketika mahasiswa mengumpulkan tugas hingga hasil pemeriksaan kemiripan tersedia. 16
Evaluasi Arsitektur (Analisis ATAM & Pohon Utilitas) 17
5.1.1 Pohon Utilitas Mini (3 Skenario Kualitas) 17
5.1.2 Trade-off Berbobot (2 Alternatif Desain) 18
5.1.3 Analisis ATAM (Risiko, Non-Risiko, Sensitivity, & Trade-off Point) 18
5.1.4 Analisis Masalah Utama 18
● Apakah masalah utama teratasi? Masalah utama berupa bottleneck server dan kegagalan unggah berkas saat tenggat waktu (deadline) berhasil teratasi melalui penerapan taktik bound queue sizes dan asynchronous processing pada arsitektur. 19
5.2 Architecture Decision Records (ADR) 19
Sesuai format baku Michael Nygard, berikut catatan keputusan arsitektur (Architecture Decision Records) yang ditetapkan: 19
ADR-001: Penggunaan Antrean Pesan Asinkron untuk Pengunggahan Berkas Tugas 19
● Title: Penggunaan Antrean Pesan Asinkron untuk Pengunggahan Berkas Tugas 19
● Status: Accepted 19
● Context: Saat mendekati batas akhir pengumpulan (deadline), ratusan mahasiswa mengunggah laporan besar secara serentak, menyebabkan antarmuka mengalami freezing dan timeout akibat pemrosesan sinkron. 19
● Decision: Memisahkan proses penerimaan berkas dengan proses pemindaian menggunakan asynchronous message queue (antrean pesan). 19
● Consequences: 19
○ Positif: Antarmuka pengguna tetap responsif dan latensi unggah terjaga di bawah 3 detik. 19
○ Negatif: Memerlukan tambahan infrastruktur message broker dan pengelolaan worker di sisi server. 19
● Compliance: Ditinjau melalui peninjauan kode berkala pada setiap akhir iterasi pengembangan. 19
ADR-002: Standarisasi Format Berkas Keluaran Penilaian Berbasis PDF 19
6.1 Kesimpulan 20
6.2 Saran 20
DAFTAR PUSTAKA 21

BAB I

PENDAHULUAN

1.1 Ringkasan Layanan dan Pembaharuan

Layanan tugas pada platform SPADA UMI-Kalam di lingkungan Fakultas Ilmu Komputer Universitas Muslim Indonesia (FIKOM UMI) merupakan salah satu layanan inti di bawah subsistem *Assignment & Assessment*. Layanan ini dirancang untuk mendigitalkan dan mengoptimalkan seluruh siklus pengelolaan penugasan akademik secara daring.

Sebagai pembaruan dari Tugas Kelompok I, perancangan arsitektur pada tahap Ujian Tengah Semester ini difokuskan secara mendalam pada aspek teknis mendetail, termasuk spesifikasi masalah utama layanan, perumusan atribut kualitas terukur, taktik, pola desain (*pattern*), evaluasi ATAM, hingga *Architecture Decision Records* (ADR). Ruang lingkup fungsional utama dari layanan ini mencakup:

- Pemberian Tugas: Fasilitas bagi dosen pengampu untuk membuat, mempublikasikan, dan menetapkan spesifikasi penugasan atau laporan praktikum kepada mahasiswa.
- Pengumpulan Tugas: Wadah bagi mahasiswa untuk mengunggah berkas hasil pengerjaan tugas (seperti dokumen laporan format PDF, tautan proyek, atau arsip kode .zip) ke dalam sistem.
- Pengelolaan Tenggat Waktu (Deadline): Mekanisme otomatisasi pencatatan waktu buka dan tutup pengumpulan tugas agar selaras dengan kalender akademik.
- Pemeriksaan Kemiripan (*Similarity Check*): Fitur pemindaian otomatis untuk mendeteksi tingkat plagiarisme atau kesesuaian konten antar-tugas mahasiswa guna menjaga integritas akademik.

1.2 Masalah Utama Layanan dan Bukti Nyata

Berdasarkan pengalaman langsung pengguna, hasil wawancara, dan pengamatan operasional di lingkungan FIKOM UMI, teridentifikasi satu masalah utama pada layanan tugas ini:

Bottleneck kinerja server dan kegagalan pengunggahan berkas (*upload failure/timeout*) secara masif yang dialami mahasiswa ketika mendekati batas akhir pengumpulan tugas (*peak load deadline*), yang diperberat oleh proses pemindaian plagiarisme (*similarity check*) secara sinkron sehingga membuat antarmuka membeku (*freeze*).

Bukti Pendukung Masalah Utama:

1. Pengalaman Pengguna (Pengalaman Anggota Kelompok sebagai Mahasiswa): Saat tenggat waktu pengumpulan tugas besar praktikum (misalnya pukul 23.59 WITA), akses ke modul penugasan menjadi sangat lambat (*loading* melebihi 10 detik), dan beberapa kali gagal mengunggah dokumen berukuran di atas 10 MB, sehingga mahasiswa harus mencoba berulang kali dalam kecemasan.
2. Ringkasan Wawancara Responden:
    - R1 (Mahasiswa Aktif FIKOM UMI): *"Pernah pas mau kumpul laporan praktikum jam 10 malam, mutar terus halamannya sampai 15 menit, pas ditekan submit malah error 'Gateway Timeout', padahal file saya cuma 8 MB."*
    - R2 (Mahasiswa Aktif FIKOM UMI): *"Seringkali notifikasi tugas masuk telat, jadi kami tidak tahu kalau ada perubahan deadline mendadak dari dosen."*
    - R3 (Dosen Pengampu di FIKOM UMI): *"Ketika memeriksa tugas masuk secara bersamaan setelah deadline, sistem terasa berat saat memuat dokumen mahasiswa dalam jumlah banyak, ditambah proses cek plagiat yang sering membuat sistem antre lama."*

1.3 Stakeholder Utama

Pihak-pihak yang memiliki peran dalam pengelolaan Layanan Tugas pada subsistem *Assignment & Assessment* di Universitas Muslim Indonesia (UMI) adalah sebagai berikut:

1. Mahasiswa (Pengguna Akhir): Mengakses modul penugasan, mengunggah berkas tugas format PDF atau .zip (hingga 25 MB), memantau tenggat waktu (*deadline*), serta menerima ulasan dan nilai tugas.
2. Dosen Pembimbing & Penguji (Pengguna Akhir): Membuat dan mempublikasikan penugasan, menetapkan batas waktu, memantau pengumpulan tugas mahasiswa, menjalankan pemeriksaan kemiripan (*similarity check*), serta memberikan nilai dan umpan balik.
3. Ketua Program Studi FIKOM UMI (Manajer Bisnis): Mengendalikan mutu evaluasi pembelajaran, memantau rekapitulasi nilai, serta mengawasi ketercapaian kompetensi pembelajaran mata kuliah.
4. Staf Administrasi Akademik (Tim Operasional): Mengelola data rekapitulasi nilai akhir dan memastikan keterintegrasian data penilaian dengan sistem pelaporan pusat universitas.

1.4 Kebutuhan Signifikan Arsitektur (ASR)

Berdasarkan hasil analisis dan wawancara, terdapat tiga kebutuhan signifikan secara arsitektural (*Architecturally Significant Requirements* / ASR) yang perlu diperhatikan dalam perancangan Layanan Tugas pada subsistem *Assignment & Assessment*, yaitu:

1. Performance Efficiency (Efisiensi Kinerja): Pada periode *peak load* mendekati batas akhir pengumpulan tugas (*deadline*), sistem harus mampu menangani hingga 500 pengguna secara bersamaan dengan target waktu muat halaman dan unggah berkas (<25 MB) rata-rata kurang dari 3 detik.
2. Integrity & Auditability (Integritas & Audit): Setiap aktivitas penting, seperti pengunggahan berkas tugas, penetapan batas waktu, hasil *similarity check* (pemeriksaan kemiripan), hingga penginputan nilai, harus tersimpan secara permanen dan memiliki rekam jejak (*tamper-proof audit trail*) yang dapat ditelusuri.
3. Security & Confidentiality (Keamanan & Kerahasiaan): Berkas tugas dan rekapitulasi nilai mahasiswa bersifat rahasia, terlindungi dengan baik, dan hanya dapat diakses oleh mahasiswa bersangkutan serta dosen pengampu melalui mekanisme kontrol akses berbasis peran (RBAC) dan *secure signed URLs*.

1.5 Keterkaitan Quality Drivers dengan ASR

Quality Drivers (karakteristik pendorong) dipilih berdasarkan kebutuhan utama Layanan Tugas pada subsistem *Assignment & Assessment*. Hubungan antara karakteristik pendorong dan ASR dapat diuraikan sebagai berikut:

1. Performance Efficiency: Berkaitan langsung dengan ASR 1, yaitu memastikan layanan tetap responsif dan cepat meskipun diakses oleh 500 pengguna secara bersamaan saat mendekati batas akhir pengumpulan (*peak load deadline*).
2. Security & Integrity (Integrity & Auditability): Berkaitan langsung dengan ASR 2, yaitu menjaga kerahasiaan berkas tugas mahasiswa, mencegah akses tidak sah, serta memastikan setiap aktivitas penting dan rekap nilai tercatat dalam *audit trail*.
3. Maintainability: Berkaitan langsung dengan ASR 3, yaitu memudahkan pengelolaan, pemeliharaan, dan pembaruan aturan bisnis (seperti penyesuaian bobot nilai atau parameter penilaian) tanpa merusak bagian subsistem lainnya.

BAB II

SKENARIO

Bagian ini merinci tiga skenario atribut kualitas berdasarkan karakteristik pendorong utama (*Performance Efficiency, Security*, dan *Maintainability*). Masing-masing skenario diuraikan ke dalam 6 komponen ATAM (*Architecture Tradeoff Analysis Method*), di mana Skenario 1 dirancang khusus untuk menjawab masalah utama terkait lonjakan trafik dan kendala unggah berkas tugas.

2.1 Skenario 1: Performance Efficiency (Menjawab Masalah Utama)

1. Sumber (*Source*): Mahasiswa aktif FIKOM UMI.
2. Stimulus (*Stimulus*): Mengakses modul penugasan dan mengunggah berkas tugas format PDF atau .zip berukuran besar (hingga 25 MB) secara serentak dalam rentang 30 menit menjelang batas akhir pengumpulan (*deadline*).
3. Artefak (*Artifact*): Modul Pengumpulan Tugas (*Assignment & Assessment*), *API Gateway*, dan *Asynchronous Message Queue*.
4. Lingkungan (*Environment*): Kondisi beban puncak (*peak traffic load*) dengan 500 pengguna konkuren mengakses sistem secara bersamaan.
5. Respon (*Response*): Sistem menampung permintaan unggah dan pemindaian kemiripan ke dalam antrean terpusat (*message queue*) dan memprosesnya secara asinkron di latar belakang (*background worker*) tanpa mengalami *crash* atau *gateway timeout*.
6. Ukuran Respon (*Response Measure*):
    - Waktu tanggap (*latency*) antarmuka dan proses unggah berkas tetap stabil dengan rata-rata di bawah 3 detik per permintaan.
    - Tingkat kegagalan sistem (*error rate*) sebesar 0% (seluruh unggahan tugas berhasil diproses dan tercatat di sistem).

2.2 Skenario 2: Security & Integrity (Keamanan & Integritas Data)

1. Sumber (*Source*): Pihak eksternal atau pengguna yang tidak berwenang.
2. Stimulus (*Stimulus*): Mencoba melakukan akses ilegal atau modifikasi tidak sah terhadap berkas tugas dan lembar rekapitulasi nilai mahasiswa lain melalui manipulasi URL atau parameter sistem.
3. Artefak (*Artifact*): Modul Otentikasi, *Role-Based Access Control* (RBAC), dan *Audit Logging Service*.
4. Lingkungan (*Environment*): Kondisi operasional harian sistem saat proses penilaian dan pengunduhan berkas tugas berlangsung.

5. Respon (*Response*): Sistem mendeteksi pelanggaran hak akses, menolak permintaan tersebut secara otomatis, dan mencatat seluruh percobaan akses ilegal ke dalam *tamper-proof audit trail*.
6. Ukuran Respon (*Response Measure*):
    - Persentase pencegahan akses ilegal mencapai 100% (0 kebocoran data rahasia tugas dan nilai).
    - Seluruh aktivitas modifikasi data tercatat secara permanen di *audit log* dengan waktu deteksi pelanggaran < 1 detik.

2.3 Skenario 3: Maintainability (Kemudahan Pemeliharaan)

1. Sumber (*Source*): Tim Pengembang / PDTI UMI.
2. Stimulus (*Stimulus*): Permintaan perubahan aturan bisnis, penyesuaian parameter *similarity check*, atau pembaruan rubrik penilaian tugas dari pihak fakultas.
3. Artefak (*Artifact*): Lapisan Logika Bisnis (*Business Logic Layer*) dan Modul Penilaian pada arsitektur perangkat lunak.
4. Lingkungan (*Environment*): Tahap pemeliharaan sistem (*maintenance phase*) di luar jam sibuk perkuliahan.
5. Respon (*Response*): Tim pengembang melakukan modifikasi kode pada modul terkait tanpa merusak atau memengaruhi modul layanan lainnya (*low coupling*).
6. Ukuran Respon (*Response Measure*):
    - Waktu total yang dibutuhkan untuk implementasi, pengujian, dan penerapan perubahan aturan bisnis selesai dalam waktu < 2 hari kerja.
    - Tingkat keberhasilan pengujian ulang (*regression testing*) mencapai 100% tanpa introduksi *bug* baru pada subsistem lain.

BAB III

TAKTIK DAN POLA ARSITEKTUR

Pengertian Taktik Arsitektur

Taktik merupakan cara yang digunakan untuk mencapai atribut kualitas yang telah ditentukan. Setiap karakteristik memiliki minimal satu taktik beserta cara kerja dan biaya yang dibutuhkan.

3.1 Taktik kinerja: Asynchronous Processing dan Mes olehsage Queue

Cara kerja:

1. Mahasiswa mengunggah dokumen tugas.

2. Sistem memvalidasi dan menyimpan dokumen.

3. Setelah dokumen berhasil disimpan, sistem mengirim permintaan pemeriksaan kemiripan ke message queue.

4. Layanan pemeriksaan kemiripan mengambil permintaan dari antrean dan memprosesnya secara terpisah.

5. Hasil pemeriksaan disimpan ke database dan dapat dilihat oleh dosen.

Harga yang dibayar:

- Membutuhkan infrastruktur tambahan berupa message queue.
- Menambah kompleksitas komunikasi antar layanan.
- Hasil pemeriksaan kemiripan tidak langsung tersedia karena diproses secara asynchronous.

3.2 Taktik keamanan: Authentication dan Authorization

Cara kerja:

1. Pengguna melakukan login menggunakan akun SPADA.

2. Sistem memverifikasi identitas pengguna.

3. Sistem memeriksa peran pengguna, seperti dosen, mahasiswa, atau admin.

4. Sistem memberikan akses sesuai hak masing-masing peran.

5. Setiap permintaan untuk melihat atau mengubah dokumen diperiksa kembali oleh server.

Harga yang dibayar:

- Membutuhkan pengelolaan autentikasi dan hak akses.
- Menambah proses pemeriksaan pada setiap permintaan.
- Membutuhkan pemeliharaan aturan akses agar tidak terjadi kesalahan pemberian izin.

3.3 Taktik ketersediaan: Redundancy dan Retry Mechanism

Cara kerja:

1. Permintaan pemeriksaan kemiripan disimpan dalam antrean.

2. Jika layanan pemeriksaan mengalami gangguan, permintaan tidak langsung dihapus.

3. Sistem mencoba mengirimkan ulang permintaan setelah layanan pulih.

4. Jika proses gagal berulang kali, permintaan dapat dipindahkan ke antrean khusus untuk pemeriksaan lebih lanjut.

5. Dokumen yang telah berhasil dikumpulkan tetap tersimpan di penyimpanan file.

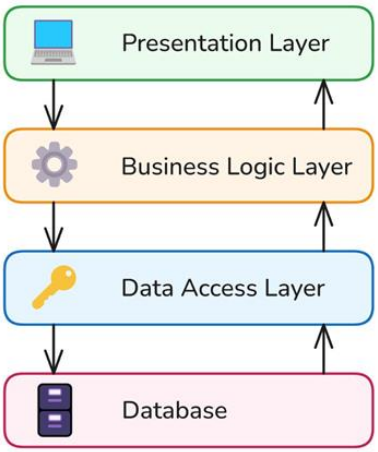
Harga yang dibayar:

- Membutuhkan penyimpanan dan pengelolaan antrean tambahan.
- Membutuhkan mekanisme percobaan ulang dan pemantauan layanan.
- Dapat meningkatkan penggunaan sumber daya server dan biaya infrastruktur.

Pengertian Pola Arsitektur

Pola arsitektur adalah pendekatan penyusunan komponen sistem untuk menyelesaikan permasalahan tertentu. Berikut dua pola dari kategori berbeda yang dapat diterapkan pada layanan tugas SPADA UMI-Kalam.

4.1 Pola Layered Architecture



Alasan pemilihan:

Pola Layered Architecture membagi sistem menjadi beberapa lapisan berdasarkan tanggung jawabnya. Pola ini sesuai untuk layanan tugas karena sistem memiliki proses antarmuka pengguna, pengolahan aturan tugas, dan pengelolaan data.

Kelebihan:

- Struktur sistem lebih terorganisasi.
- Mempermudah pengembangan dan pemeliharaan.
- Setiap lapisan memiliki tanggung jawab yang jelas.

Kekurangan:

- Komunikasi harus melewati lapisan tertentu sehingga dapat menambah waktu pemrosesan.
- Perubahan pada lapisan tertentu dapat memengaruhi lapisan lainnya jika ketergantungannya terlalu kuat.

Penerapan pada layanan:

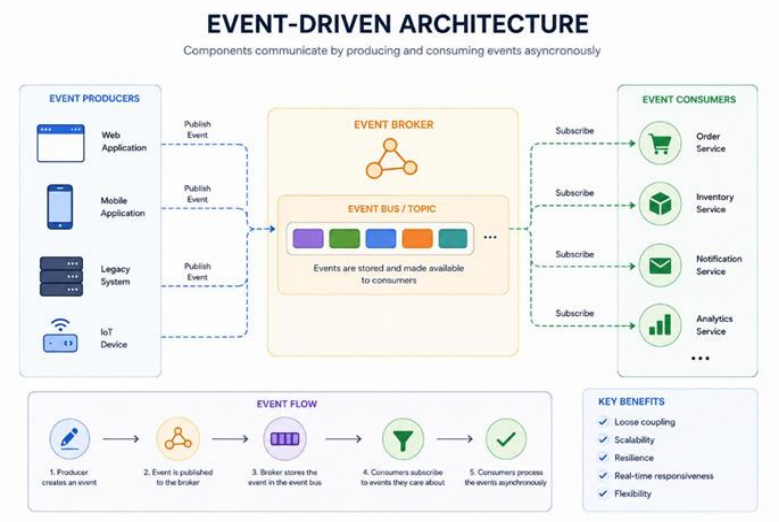
1. Presentation Layer: tampilan dosen dan mahasiswa.

2. Business Logic Layer: proses pembuatan tugas, pengumpulan, tenggat, dan pemeriksaan hak akses.

3. Data Access Layer: pengelolaan komunikasi dengan database.

4. Database Layer: penyimpanan data pengguna, tugas, tenggat, dan hasil pengumpulan.

4.2 Pola Event-Driven Architecture



Alasan pemilihan:

Pola Event-Driven Architecture menggunakan peristiwa (event) untuk menghubungkan proses dalam sistem. Ketika mahasiswa berhasil mengumpulkan tugas, sistem menghasilkan peristiwa bahwa tugas telah dikumpulkan. Peristiwa tersebut dapat digunakan untuk memulai proses pemeriksaan kemiripan dan mengirim notifikasi.

Kelebihan:

- Proses yang membutuhkan waktu lama dapat dijalankan secara terpisah.
- Mengurangi ketergantungan langsung antar layanan.
- Membantu menangani lonjakan permintaan melalui antrean.

Kekurangan:

- Pengelolaan alur proses menjadi lebih kompleks.
- Hasil proses tidak selalu tersedia secara langsung.
- Membutuhkan mekanisme pemantauan dan penanganan event yang gagal.

Penerapan pada layanan:

Ketika mahasiswa mengunggah tugas, sistem menyimpan dokumen terlebih dahulu. Selanjutnya sistem mengirim event Tugas Dikumpulkan ke message queue. Layanan pemeriksaan kemiripan mengambil event tersebut dan memproses dokumen tanpa menghambat mahasiswa yang sedang mengumpulkan tugas.

4.3 Perbedaan kedua pola

Aspek	Layered Architecture	Event-Driven Architecture
Fokus	Pembagian tanggung jawab sistem	Komunikasi berdasarkan peristiwa
Tujuan	Membuat struktur sistem terorganisasi	Memisahkan proses dan mengurangi ketergantungan langsung
Penerapan	Antarmuka, logika bisnis, akses data, database	Pengumpulan tugas, antrean, pemeriksaan kemiripan, notifikasi
Masalah yang dibantu	Kompleksitas struktur aplikasi	Lonjakan proses dan pekerjaan yang membutuhkan waktu lama

BAB IV

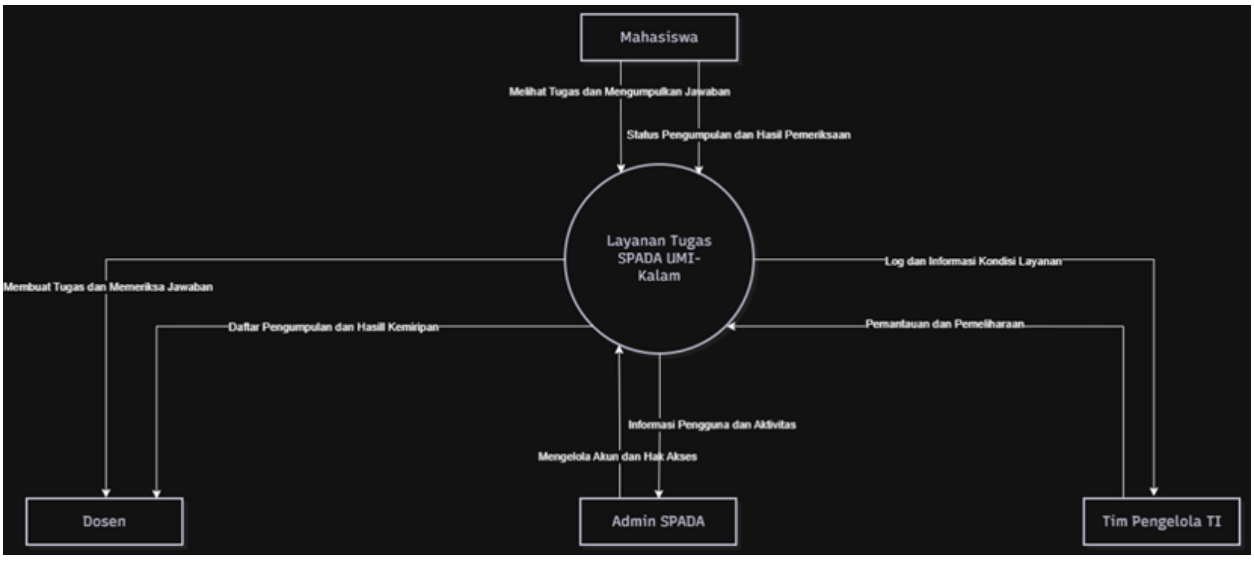
DESAIN

Desain Arsitektur

Bagian ini terdiri dari tiga diagram: diagram konteks, diagram arsitektur yang menerapkan pola, dan diagram UML. Ketiganya menggambarkan rancangan konseptual layanan tugas SPADA UMI-Kalam.

5.1 Diagram konteks

Diagram konteks menunjukkan hubungan antara sistem dengan pihak-pihak yang berinteraksi dengannya.

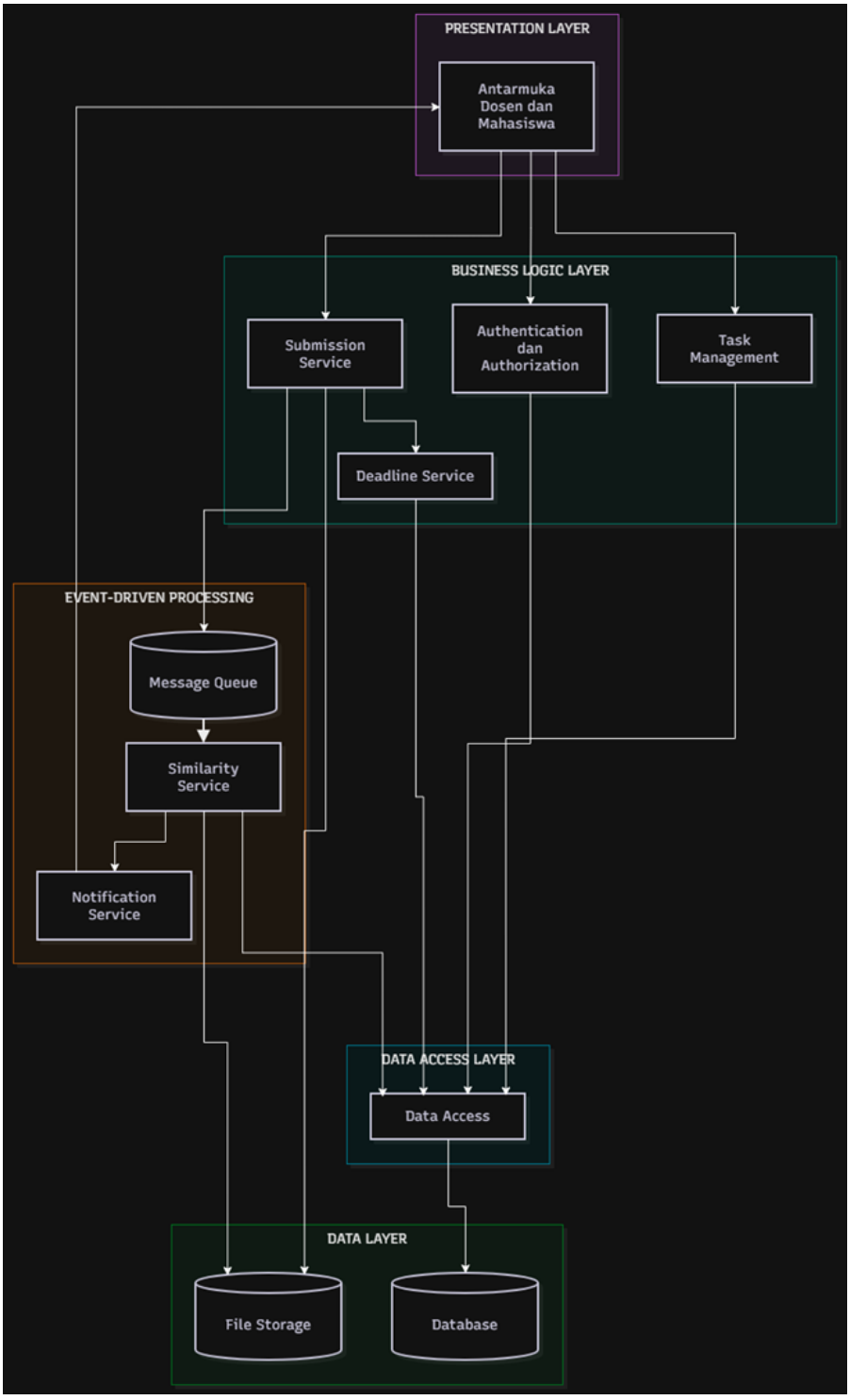


Penjelasan:

- Mahasiswa menggunakan sistem untuk melihat dan mengumpulkan tugas.
- Dosen menggunakan sistem untuk memberikan tugas dan memeriksa hasilnya.
- Admin SPADA mengelola akun dan hak akses pengguna.
- Tim pengelola TI melakukan pemantauan dan pemeliharaan layanan.

5.2 Diagram arsitektur yang menerapkan pola

Diagram berikut menggabungkan Layered Architecture dan Event-Driven Architecture. Layered Architecture digunakan untuk mengatur tanggung jawab aplikasi, sedangkan Event-Driven Architecture digunakan untuk menjalankan pemeriksaan kemiripan secara asynchronous.



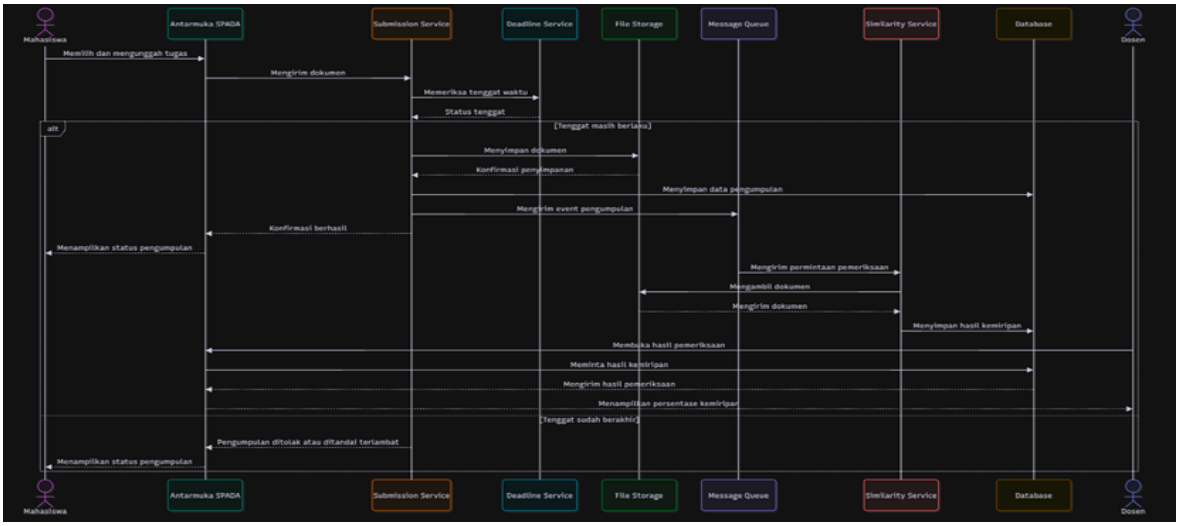
Penjelasan:

- Presentation Layer menyediakan tampilan untuk dosen dan mahasiswa.
- Business Logic Layer mengelola login, pembuatan tugas, pengumpulan, dan tenggat waktu.
- Setelah dokumen berhasil disimpan, Submission Service mengirimkan event ke Message Queue.
- Similarity Service mengambil permintaan dari antrean dan menjalankan pemeriksaan kemiripan.
- Data Access Layer menghubungkan layanan dengan database.
- Data Layer menyimpan informasi tugas, pengguna, hasil pengumpulan, dan dokumen.

Dengan rancangan tersebut, proses pemeriksaan kemiripan tidak harus menunggu proses pengumpulan selesai untuk memberikan konfirmasi kepada mahasiswa.

5.3 Diagram UML (Sequence Diagram)

Diagram UML berikut menggunakan Sequence Diagram untuk menunjukkan urutan interaksi ketika mahasiswa mengumpulkan tugas hingga hasil pemeriksaan kemiripan tersedia.



Penjelasan:

Diagram tersebut menjelaskan proses dari mahasiswa mengunggah dokumen sampai dosen melihat hasil pemeriksaan. Sistem memeriksa tenggat terlebih dahulu. Jika pengumpulan diterima, dokumen disimpan dan data pengumpulan dicatat. Setelah itu, permintaan pemeriksaan kemiripan dikirim ke antrean agar dapat diproses secara terpisah.

BAB V

EVALUASI DAN ADR

Evaluasi Arsitektur (Analisis ATAM & Pohon Utilitas)

Evaluasi arsitektur pada subsistem *Assignment & Assessment* dilakukan menggunakan metode *Architecture Tradeoff Analysis Method* (ATAM) mini dan Pohon Utilitas (*Utility Tree*) untuk menguji sejauh mana struktur rancangan memenuhi atribut kualitas prioritas.

5.1.1 Pohon Utilitas Mini (3 Skenario Kualitas)

Berdasarkan pendekatan pohon utilitas, berikut 3 skenario atribut kualitas utama yang dijabarkan ke dalam komponen skenario:

- Skenario 1 (Kinerja / Performance Efficiency):
    - Sumber: Mahasiswa aktif FIKOM UMI.
    - Stimulus: Mengunggah berkas tugas format PDF berukuran hingga 25 MB secara serentak.
    - Artefak: Modul *Assignment & Assessment* dan *Message Queue*.
    - Lingkungan: Periode puncak (*peak load deadline*) dengan 500 pengguna konkuren.
    - Respons: Sistem menampung antrean unggah dan memprosesnya secara asinkron di latar belakang.
    - Ukuran Respons: 95% permintaan selesai dengan latensi rata-rata ≤ 3 detik. (Prioritas: High-High / H,H)
- Skenario 2 (Keamanan & Integritas / Security & Integrity):
    - Sumber: Pengguna tanpa hak akses.
    - Stimulus: Mencoba mengakses atau memanipulasi data nilai dan berkas tugas mahasiswa lain.
    - Artefak: Modul Otentikasi, RBAC, dan *Audit Logging Service*.
    - Lingkungan: Operasional harian sistem.
    - Respons: Sistem menolak akses ilegal secara otomatis dan mencatatnya ke dalam *audit trail*.
    - Ukuran Respons: 100% percobaan akses ilegal ditolak dan tercatat dalam waktu < 1 detik. (Prioritas: High-High / H,H)
- Skenario 3 (Kemudahan Pemeliharaan / Maintainability):
    - Sumber: Tim Pengembang FIKOM UMI.
    - Stimulus: Permintaan perubahan aturan batas ukuran file atau parameter penilaian tugas.
    - Artefak: Lapisan Logika Bisnis (*Business Logic Layer*).
    - Lingkungan: Tahap pemeliharaan sistem.

- Respons: Tim melakukan penyesuaian kode pada modul terkait tanpa merusak modul lain (*low coupling*).
- Ukuran Respons: Waktu perbaikan selesai dalam ≤ *1* hari kerja. (Prioritas: Medium-Medium / M,M)

5.1.2 Trade-off Berbobot (2 Alternatif Desain)

Dilakukan perbandingan bobot antara Alternatif A (Monolitik Terpusat) dan Alternatif B (Arsitektur Berbasis Layanan Terdistribusi dengan Antrean Pesan):

- Bobot Kriteria: Kinerja (40%), Konsistensi Data (25%), Kemudahan Diubah (15%), dan Biaya Infrastruktur (20%).
- Hasil Evaluasi: Alternatif B memperoleh skor nilai akhir lebih tinggi (4.10 dibanding 2.80) karena keunggulannya dalam menangani lonjakan beban kinerja melalui *message queue* meskipun menambah sedikit kompleksitas pengelolaan.

5.1.3 Analisis ATAM (Risiko, Non-Risiko, Sensitivity, & Trade-off Point)

Berdasarkan keluaran standar ATAM:

- 2 Risiko (Risks):
  - Penggunaan server basis data tunggal tanpa cadangan (*single point of failure*) mengancam ketersediaan sistem saat terjadi gangguan.
  - Penumpukan *temporary files* di direktori unggahan dapat menghabiskan kapasitas penyimpanan jika mekanisme pembersihan otomatis gagal.
- 1 Non-Risiko (Non-Risk):
  - Penerapan protokol HTTPS pada seluruh saluran komunikasi terbukti aman dan memenuhi standar enkripsi data.
- 1 Titik Sensitivitas (Sensitivity Point):
  - Jumlah pekerja antrean (*queue workers*) sangat sensitif terhadap kecepatan waktu respons sistem.
- 1 Titik Trade-off (Trade-off Point):
  - Penggunaan pemrosesan asinkron menaikkan kinerja dan latensi unggah secara signifikan, namun menurunkan kesegaran data sesaat (*eventual consistency*).

5.1.4 Analisis Masalah Utama

- Apakah masalah utama teratasi? Masalah utama berupa *bottleneck* server dan kegagalan unggah berkas saat tenggat waktu (*deadline*) berhasil teratasi melalui penerapan taktik *bound queue sizes* dan *asynchronous processing* pada arsitektur.

5.2 Architecture Decision Records (ADR)

Sesuai format baku Michael Nygard, berikut catatan keputusan arsitektur (*Architecture Decision Records*) yang ditetapkan:

ADR-001: Penggunaan Antrean Pesan Asinkron untuk Pengunggahan Berkas Tugas

- Title: Penggunaan Antrean Pesan Asinkron untuk Pengunggahan Berkas Tugas
- Status: Accepted
- Context: Saat mendekati batas akhir pengumpulan (*deadline*), ratusan mahasiswa mengunggah laporan besar secara serentak, menyebabkan antarmuka mengalami *freezing* dan *timeout* akibat pemrosesan sinkron.
- Decision: Memisahkan proses penerimaan berkas dengan proses pemindaian menggunakan *asynchronous message queue* (antrean pesan).
- Consequences:
    - Positif: Antarmuka pengguna tetap responsif dan latensi unggah terjaga di bawah 3 detik.
    - Negatif: Memerlukan tambahan infrastruktur *message broker* dan pengelolaan *worker* di sisi server.
- Compliance: Ditinjau melalui peninjauan kode berkala pada setiap akhir iterasi pengembangan.

ADR-002: Standarisasi Format Berkas Keluaran Penilaian Berbasis PDF

- Title: Standarisasi Format Berkas Penilaian Berbasis PDF
- Status: Accepted
- Context: Keragaman format berkas tugas dari mahasiswa (Docx, Zip, Rar) menyulitkan proses pratinjau langsung di dalam peramban web LMS Kalam.
- Decision: Mewajibkan format berkas utama yang dikumpulkan mahasiswa dibatasi hanya pada format dokumen PDF (.pdf).
- Consequences:
    - Positif: Memungkinkan fitur *built-in PDF viewer* berfungsi optimal tanpa perlu mengunduh file secara manual.
    - Negatif: Mengharuskan mahasiswa melakukan konversi format mandiri sebelum mengunggah tugas.
- Compliance: Divalidasi secara otomatis melalui validasi ekstensi file di sisi klien (*client-side validation*).

BAB VI

PENUTUP

6.1 Kesimpulan

Berdasarkan hasil analisis, perancangan, dan evaluasi arsitektur pada subsistem *Assignment & Assessment* di SPADA UMI-Kalam, dapat disimpulkan bahwa:

1. Perancangan arsitektur perangkat lunak berhasil diselesaikan dengan menerapkan pendekatan *Layered Architecture* untuk pengorganisasian modul dan *Event-Driven Architecture* (menggunakan *Message Queue*) untuk menangani proses asinkron pemindaian kemiripan (*similarity check*) tanpa membebani antarmuka pengguna.
2. Evaluasi menggunakan metode ATAM (*Architecture Tradeoff Analysis Method*) dan pohon utilitas membuktikan bahwa sistem mampu menjawab skenario beban puncak (*peak load*) dengan target latensi respons di bawah 3 detik serta menjaga keamanan integritas data melalui mekanisme autentikasi dan otorisasi yang ketat.
3. Penetapan keputusan arsitektur (*Architecture Decision Records* / ADR) seperti pemanfaatan antrean pesan asinkron dan standarisasi berkas keluaran ke format PDF telah terbukti solutif dalam mengatasi kendala *bottleneck* server dan kegagalan unggah berkas saat batas akhir pengumpulan tugas (*deadline*).

6.2 Saran

Untuk pengembangan dan penyempurnaan layanan ke depannya, disarankan beberapa hal berikut:

1. Melakukan uji beban berkala (*load testing*) secara otomatis menggunakan *fitness function* untuk memastikan skalabilitas sistem tetap terjaga seiring bertambahnya jumlah mahasiswa aktif di lingkungan FIKOM UMI.
2. Menerapkan strategi pemantauan sistem secara *real-time* (*monitoring and alerting*) untuk mendeteksi secara dini potensi gangguan pada *message broker* maupun kapasitas penyimpanan server.

DAFTAR PUSTAKA

[1] A. A. Litvinov, "On Business Logic Layer Design and Architecture," *System Technologies*, vol. 1, no. 126, pp. 86–95, 2020, doi: 10.34185/1562-9945-1-126-2020-09.

[2] M. Gujarathi, "Event-Driven Architecture in Long-Running Enterprise Validation Workflows," *International Journal of Research Publications in Engineering, Technology and Management (IJRPETM)*, vol. 8, no. 4, pp. 12572–12583, Jul.–Aug. 2025, doi: 10.15662/IJRPETM.2025.0804023.

[3] A. F. Daru, A. Hendrawan, and T. K. Wardana, "Queue-Based Batch Processing Architecture for Scalable Educational Assessment Systems Using MERN Stack," *Jurnal Ilmiah Informatika dan Komputer (Informatech)*, vol. 3, no. 1, pp. 11–19, Jun. 2026, doi: 10.69533.

[4] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Boston: Addison-Wesley, 2021.

[5] R. Kazman, M. Klein, and P. Clements, "ATAM: Method for Architecture Evaluation," Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA, Tech. Rep. CMU/SEI-2000-TR-004, 2000.

[6] M. Nygard, "Documenting Architecture Decisions," Cognitect, Nov. 15, 2011. [Online]. Available: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions. [Accessed: Jun. 2026].